

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

Answer: I used the **full MNIST dataset** — 60,000 training images and 10,000 test images. The first thing I did was divide every pixel by 255 so everything sits between 0 and 1. For the fully connected model I had to flatten each image into 784 numbers, but for the convolutional ones I kept the shape as $28 \times 28 \times 1$ so the spatial structure

stays intact. I also reused the 10,000 test images as my validation set, which is what I used to draw the loss curves you'll see later.

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

Original image
Reconstructed image
Reconstruction error

For the denoising model, display:

Original clean image
Noisy image
Denoised reconstruction

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

Answer: The original-vs-reconstructed pairs are in Figures 1, 5 and 13. The clean/noisy/denoised sets are in Figures 7, 8, 8 plus the salt-and-pepper ones. The fresh images the VAE cooked up are in Figure 15. And for the reconstruction error, I went through every single test image and computed its error, which you can see in Figures 19 and 20.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

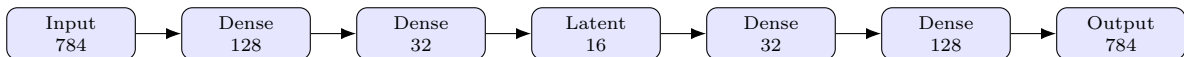
7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.

Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.

Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

Answer: I followed this architecture almost exactly as it's written. So the flow goes $784 \rightarrow 128 \rightarrow 32 \rightarrow 16 \rightarrow 32 \rightarrow 128 \rightarrow 784$, and when I counted up the weights it came to **211,040** parameters. ReLU everywhere in the hidden layers, sigmoid on the final output layer. Training was 20 epochs with Adam at a learning rate of 0.001, batch size 128, and binary cross-entropy as the loss.

8. Required Analysis of the Basic Autoencoder

Plot 1: Original versus Reconstructed Images

Display at least 10 test images in a grid.

For every image, show:

Original | Reconstruction.

Students must comment on:

- preservation of digit shape;
- loss of fine details;
- blurred regions;
- differences between easy and difficult samples.

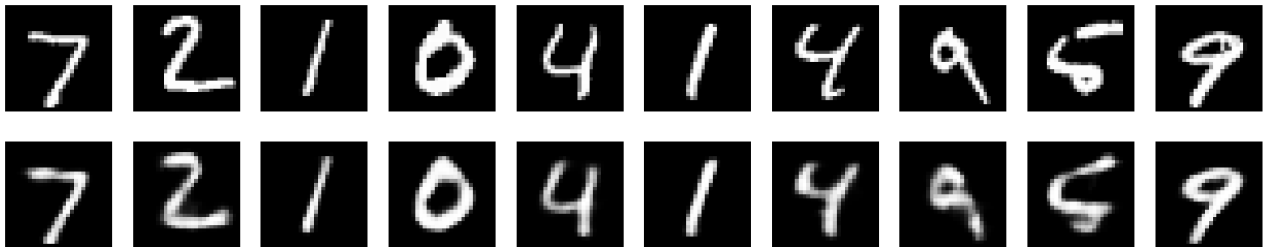


Figure 1: Plot 1: Original test images (top row) and fully connected AE reconstructions (bottom row).

Answer: So the top row is the actual digit and the bottom row is whatever the autoencoder spat out. The good news is that the **overall shape of every digit is preserved** — I can still read off 7, 2, 1, 0, 4, 1, 4, 9, 5, 9 without any trouble. The bad news is that the output is clearly **blurry**, and the thin strokes end up looking thicker and kind of smudged. Easy digits like 1, 0 and 7 come out almost identical to the original, which makes sense. But the harder ones like 2, 4, 5 and 9 lose all the small details — the top of the 4 becomes really weak, the 9 is basically a blur, and the 5 loses that little top line. I think this is just what happens when you force 784 pixels through a bottleneck of only 16 numbers.

Plot 2: Training and Validation Reconstruction Loss

- X-axis: Epoch
- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss

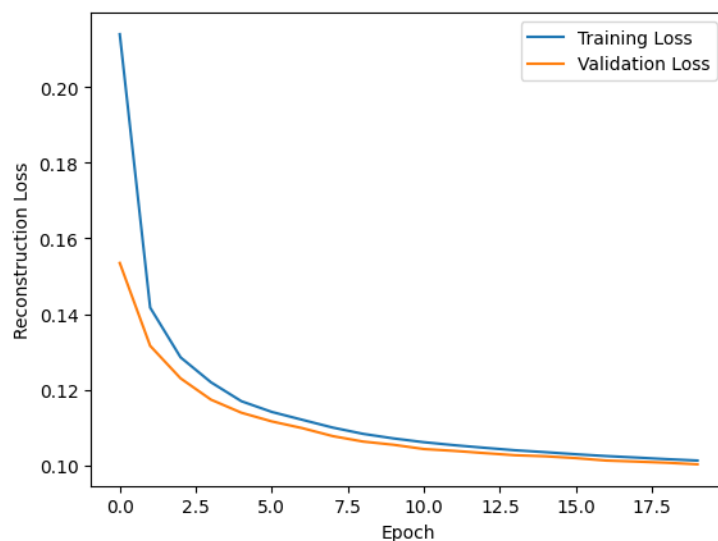


Figure 2: Plot 2: Training and validation reconstruction loss of the fully connected AE.

Required inference: Determine whether the model converges and whether there is evidence of overfitting.

Answer: Looking at the 20 epochs: both curves crash down really fast in the first 4 or 5 epochs, and after that they just kind of crawl down and flatten out. Training loss dropped from 0.2140 to 0.1013, validation from 0.1535 to 0.1003. The important thing is the two lines are basically sitting on top of each other the whole time — there's no

gap. So the model definitely **converges** and I see **zero signs of overfitting**. One thing to note: the loss never hits zero, and that's actually expected because binary cross-entropy on MNIST pixels just doesn't have a zero minimum.

9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Structural Similarity

Calculate SSIM between the original and reconstructed images.

The implementation may use:

<https://scikit-image.org/docs/stable/api/skimetrics.html>

Report:

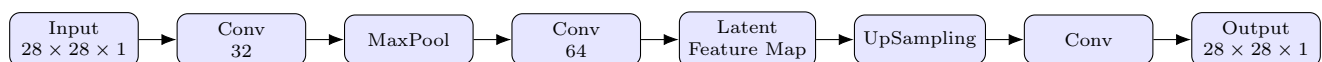
Metric	Value
Test MSE	0.01271
Test MAE	0.03927
Mean SSIM	0.8593

Answer: MSE came out to 0.0127 and MAE to 0.0393, which is pretty low — it means on average each pixel is only off by a tiny amount. SSIM is 0.86, so structurally the reconstruction is about 86% similar to the original. That's decent, but not amazing, and you can definitely see why when you look at the blur in Plot 1.

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.

The conceptual architecture is:



A possible implementation is:

```

Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
  
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

Answer: I used exactly the layers shown above, with UpSampling2D (didn't switch to transposed convolutions). So the spatial size shrinks $28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7$, which means the bottleneck is a $7 \times 7 \times 64$ feature map. Funny thing — this model only has **74,497** parameters, way fewer than the FC version. Trained for 20 epochs with Adam and binary cross-entropy. Training loss went from 0.1031 down to 0.0632, validation from 0.0733 down to 0.0628. Again the two are basically touching, so no overfitting.

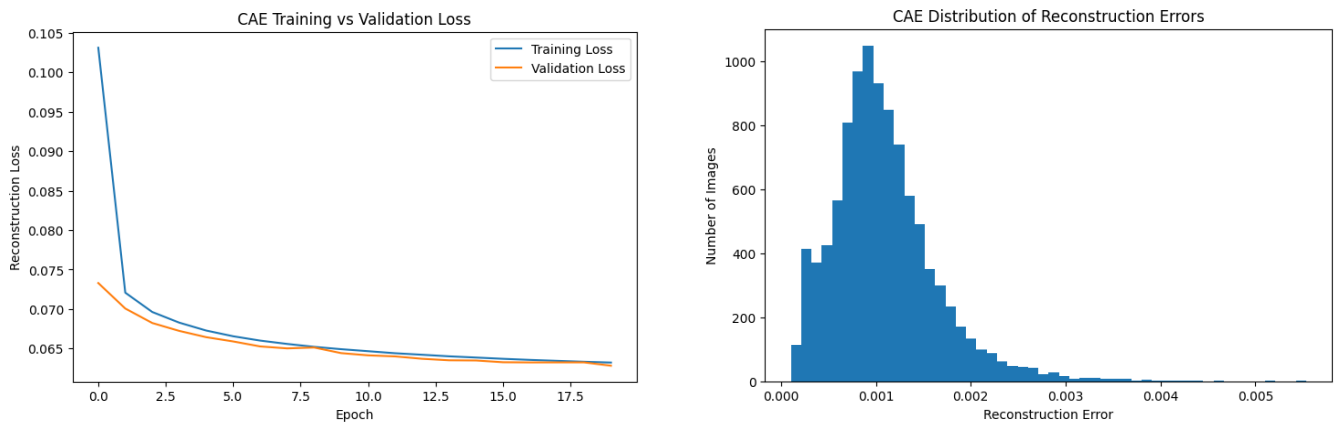


Figure 3: Left: training and validation loss of the CAE. Right: distribution of reconstruction errors of the CAE.

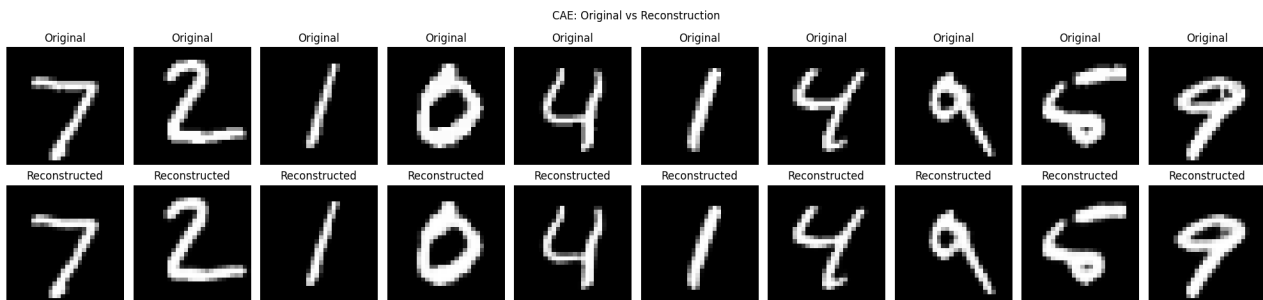


Figure 4: Original (top) and CAE reconstruction (bottom) for 10 test images.

11. Comparing Fully Connected and Convolutional Autoencoders

Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.01271	0.03927	0.8593	211,040
Convolutional AE	0.001086	0.009741	0.9897	74,497

Plot 3: Reconstruction Comparison

Display:

Original $\rightarrow$ FC-AE Reconstruction $\rightarrow$ CAE Reconstruction.

Required inference:

Discuss how preserving spatial locality through convolution affects image reconstruction.

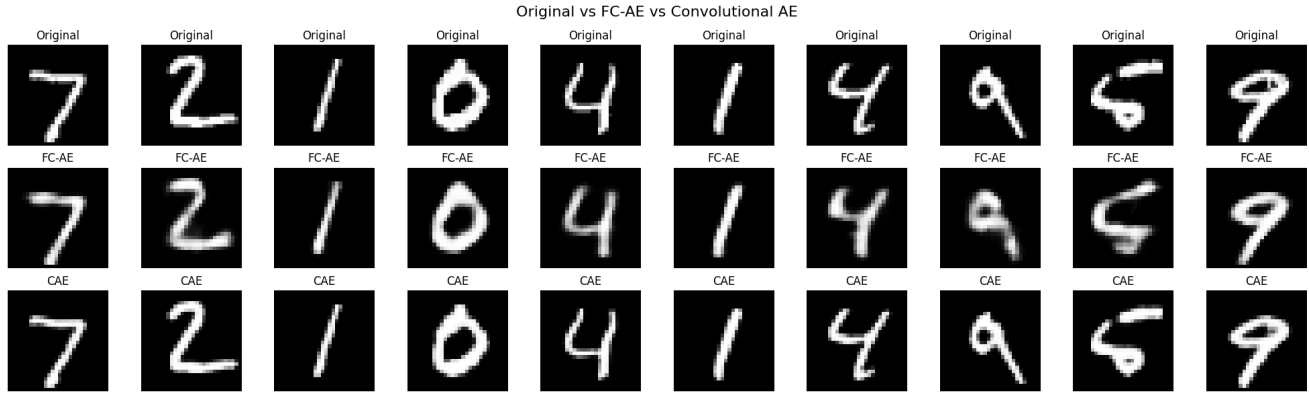


Figure 5: Plot 3: Original (top), FC-AE (middle) and CAE (bottom) reconstructions.

Answer: This figure is honestly the best way to see the difference. Same 10 digits, but the CAE images (bottom row) are sharp and basically look like the originals, while the FC-AE ones (middle row) are noticeably blurry — especially for 2, 4, 9 and 5. The metrics back this up too: MSE drops from 0.0127 to 0.0011, so the CAE is roughly 12 times better, and SSIM jumps from 0.859 to 0.990. And here’s the kicker — the CAE did this with **far fewer parameters**, only 74,497 vs 211,040. The reason, I think, is that a convolution looks at neighbouring pixels as a group, so the strokes and edges of the digit stay intact, and the same small filter gets reused across the whole image. The FC-AE flattens everything and completely forgets which pixels used to be next to each other. On top of that, the CAE bottleneck ($7 \times 7 \times 64 = 3136$ values) is way bigger than the FC-AE’s 16 numbers, so a lot more information survives. Bottom line: convolution really pays off here.

Note: the FC-AE and CAE were trained once for Section 11 and once again (fresh weights) for Sections 21 and 25, so the numbers are very slightly different between the tables.

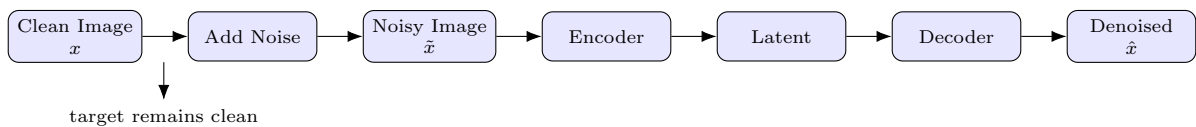
12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image.

The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

Answer: I tried both noise types. For Gaussian I used $\sigma = 0.1, 0.2, 0.3$ and clipped everything to $[0, 1]$ afterwards so the values don't go out of range. For salt-and-pepper I used $p = 0.05, 0.10, 0.20$, where I made half the corrupted pixels go to 0 (that's the pepper) and the other half go to 1 (that's the salt). I trained a fresh denoising CAE for each noise level, feeding the noisy image as input and keeping the clean image as the target.

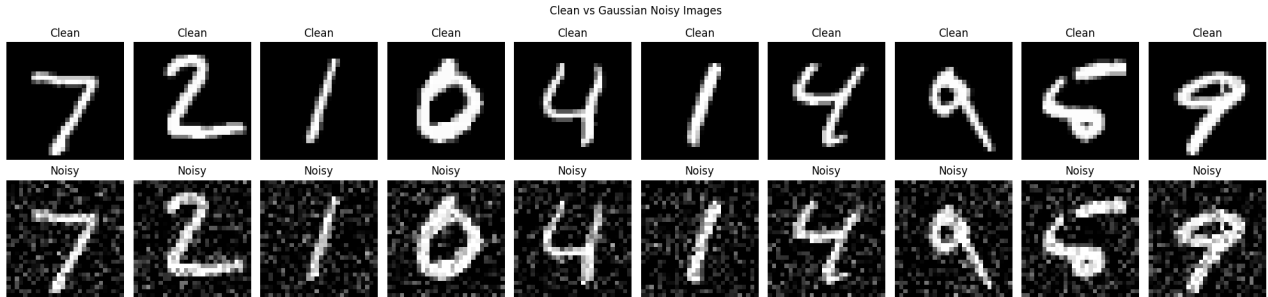


Figure 6: Clean images (top) and Gaussian noisy images with $\sigma = 0.2$ (bottom).

14. Required Denoising Plots

Plot 4: Clean vs. Noisy vs. Denoised

For at least 10 images, display:

Clean | Noisy | Denoised.

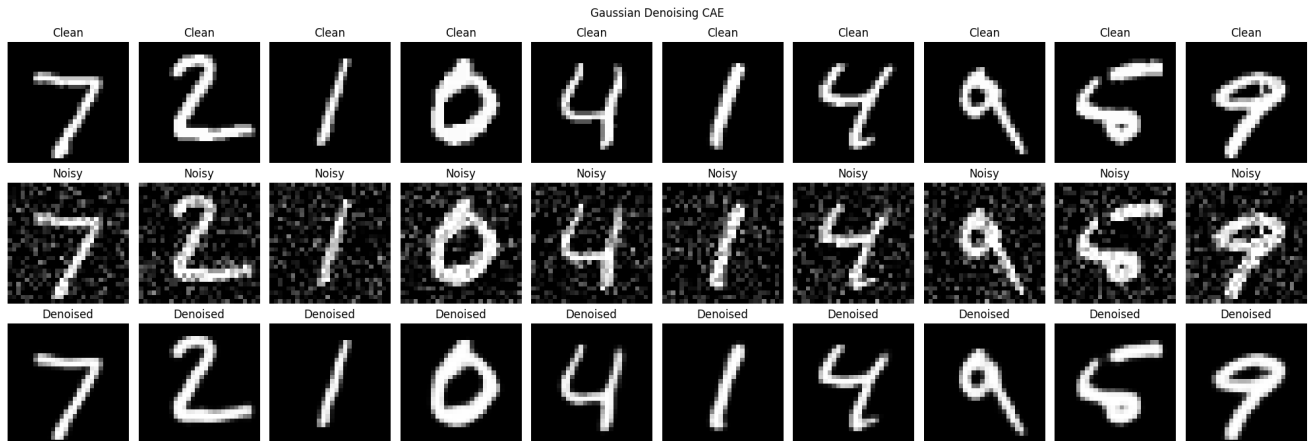


Figure 7: Plot 4: Clean, noisy and denoised images for Gaussian noise $\sigma = 0.2$.

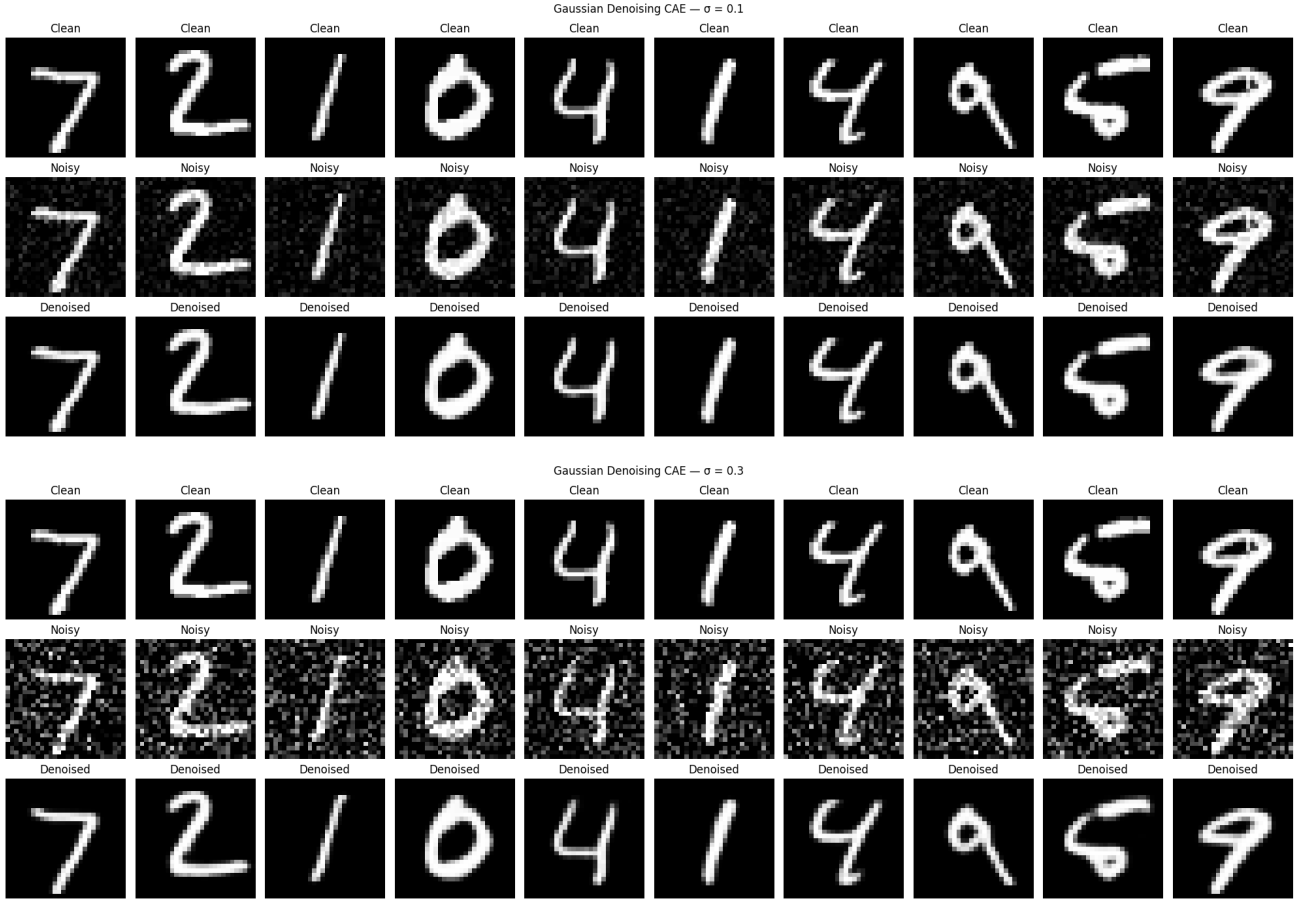


Figure 8: Clean, noisy and denoised images for Gaussian noise $\sigma = 0.1$ (top) and $\sigma = 0.3$ (bottom).

Plot 5: Noise Level vs. Reconstruction Quality

For each noise level, calculate:

$$MSE, \quad MAE, \quad SSIM.$$

Report:

Noise Level	MSE	MAE	SSIM
0.1	0.001438	0.011571	0.9819
0.2	0.002695	0.016038	0.9650
0.3	0.004483	0.020824	0.9438

Salt-and-pepper noise (same denoising architecture):

Noise Level p	MSE	MAE	SSIM
0.05	0.002025	0.013152	0.9783
0.10	0.002659	0.015005	0.9707
0.20	0.003999	0.018707	0.9566

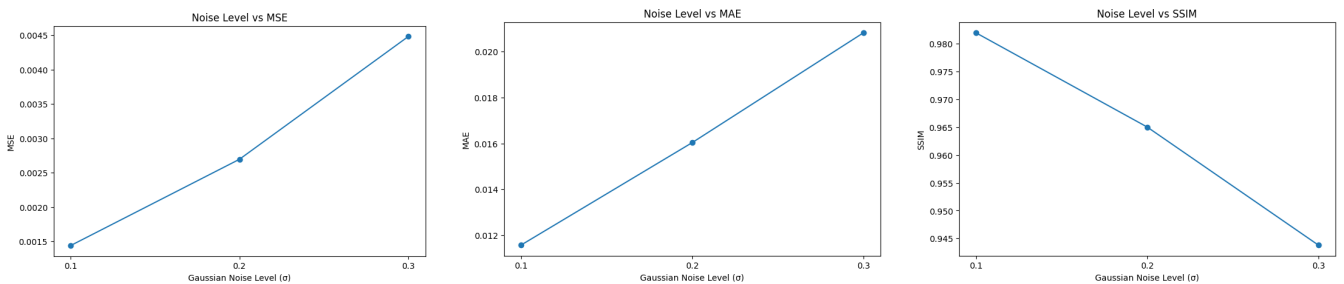


Figure 9: Plot 5: Gaussian noise level σ versus MSE (left), MAE (middle) and SSIM (right).

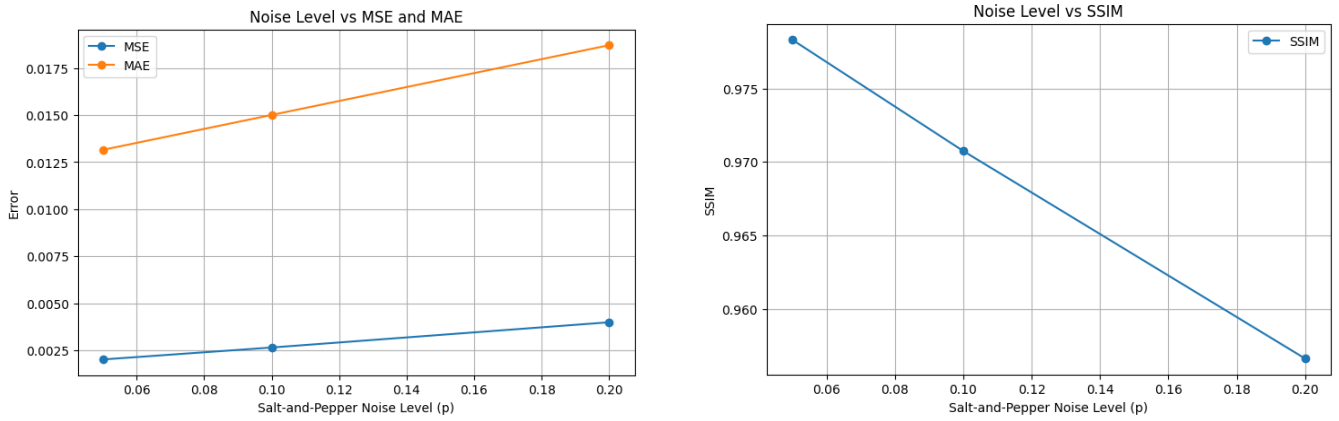


Figure 10: Salt-and-pepper noise level p versus MSE and MAE (left) and SSIM (right).

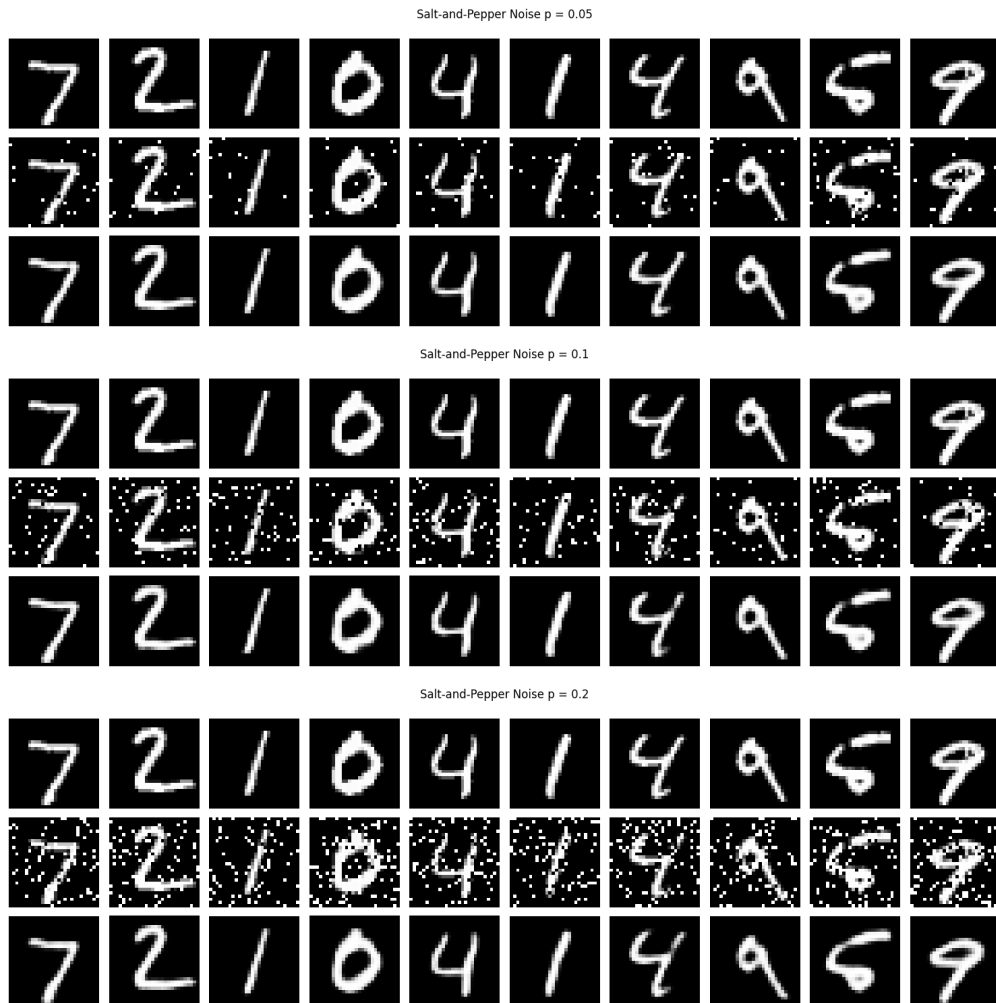


Figure 11: Salt-and-pepper denoising for $p = 0.05, 0.10$ and 0.20 (rows in each image: clean, noisy, denoised).

Required inference:

Explain how increasing corruption affects reconstruction quality and identify whether the denoising model is able to recover the major structure of the original image.

Answer: In Plot 4 you can see the middle row is just covered in noise dots, but the bottom row brings back clean digits — that's the satisfying part. As expected, when I crank the noise from $\sigma = 0.1$ up to 0.3 , the numbers get worse across the board: MSE climbs ($0.0014 \rightarrow 0.0027 \rightarrow 0.0045$), MAE climbs ($0.0116 \rightarrow 0.0160 \rightarrow 0.0208$), and SSIM slides down ($0.982 \rightarrow 0.965 \rightarrow 0.944$). That makes sense, really — the more noise you throw at it, the less real pixel information is left for the decoder to work with. But the drop is surprisingly gentle, and even at $\sigma = 0.3$ the SSIM is still sitting above 0.94 . Salt-and-pepper shows the same story (SSIM $0.978, 0.971, 0.957$). So yes, the denoising model does a solid job of **recovering the main structure** of the digits; only some really fine stroke details get sacrificed.

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

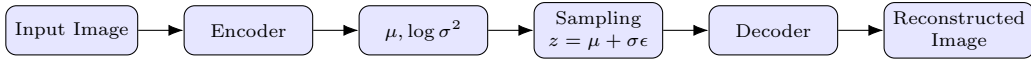
$$q_\phi(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

17. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

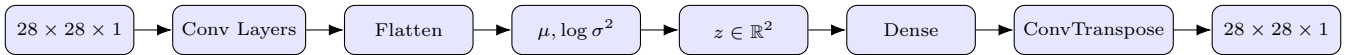
The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \rightarrow 28 \times 28 \times 1.$$

A compact architecture is:



Answer: My encoder goes $\text{Conv}(32, \text{stride } 2) \rightarrow \text{Conv}(64, \text{stride } 2) \rightarrow \text{Flatten}$ (which gives 3136 values) $\rightarrow \text{Dense}(16)$, and then two parallel Dense layers of size 2 produce μ and $\log \sigma^2$. The sampling step is a custom layer that does $z = \mu + \sigma\epsilon$ — that’s the reparameterization trick. On the decoder side it’s $\text{Dense}(3136) \rightarrow \text{Reshape to } 7 \times 7 \times 64 \rightarrow \text{three Conv2DTranspose layers} \rightarrow 28 \times 28 \times 1$ with sigmoid. Parameters came out to 69,076 for the encoder and 65,089 for the decoder, so the whole VAE is **134,165** parameters. Loss is binary cross-entropy summed over all 784 pixels plus the KL term. Trained 30 epochs with Adam (0.001 lr, batch 128) and it took about 115 seconds.

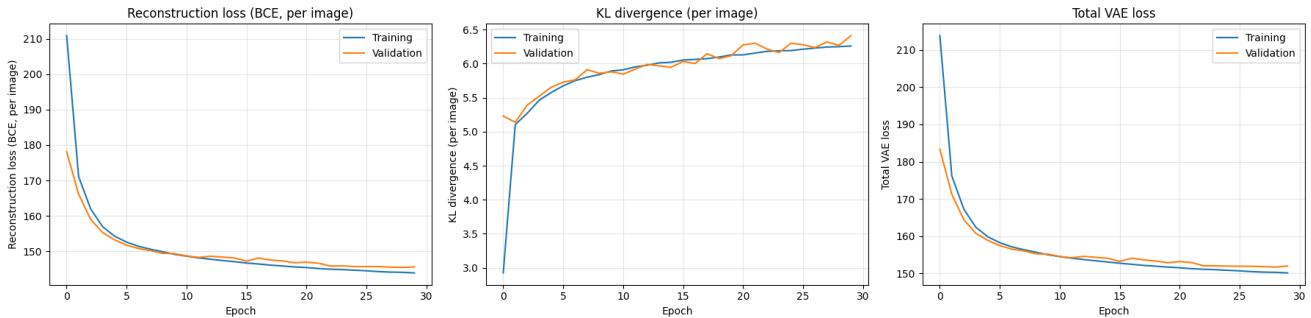


Figure 12: Plot: VAE training and validation reconstruction loss (left), KL loss (middle) and total loss (right).

Answer: The reconstruction loss (per image) drops from around 211 to 144 on training data and from 178 to 146 on validation. Meanwhile the KL loss actually goes *up*, from about 3 to 6.3, which is a good sign — it means the encoder is actually using the latent space more as training goes on. After roughly epoch 20 the validation loss just plateaus around 152, while training keeps inching down (150.1 at the very end). The gap is only about 2, so nothing to worry about — **no serious overfitting** and the model has clearly converged.

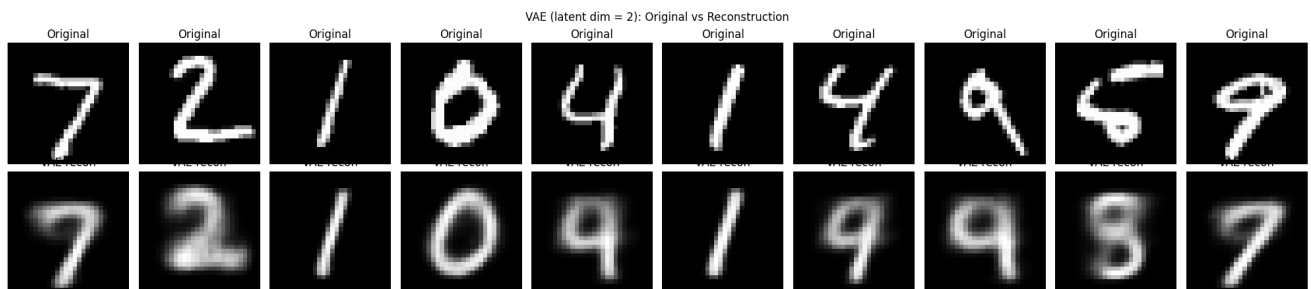


Figure 13: VAE (latent dimension 2): original test images (top) and reconstructions (bottom).

Answer: The VAE reconstructions are noticeably blurrier than the plain AE ones. Simple digits like 1, 0 and 7 come out fine, but it messes up the harder ones: the 4 turns into a 9, the second 4 also becomes a 9, the 5 turns into an 8, and the last 9 becomes a 7. Not great, but honestly kind of expected — you’re trying to cram an entire image into just two numbers.

18. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.

Plot 6: VAE Latent Space

- X-axis: z_1
- Y-axis: z_2
- Color/marker category: digit label

The labels are used only for visualization; they are not supplied to the VAE during training.

Required inference:

Students should describe whether samples belonging to the same digit occupy similar regions and whether different digits overlap.

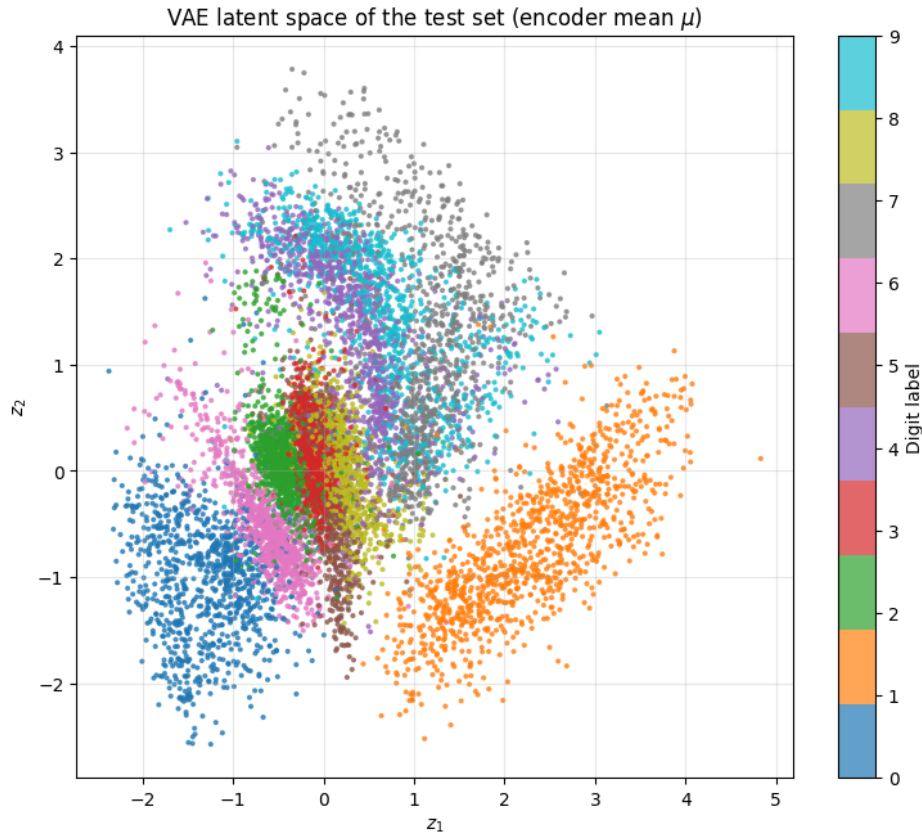


Figure 14: Plot 6: Two-dimensional latent space of the VAE for the 10,000 test images, coloured by digit label.

Answer: Every dot is a test image and the colour tells you which digit it is. What’s cool is that images of the same digit naturally cluster together. Digit 1 (orange) hangs out in the bottom right and digit 0 (blue) in the bottom left — those two are nicely separated. Digit 6 (pink) sits between 0 and the middle group. But honestly, a lot of it is a big messy overlap: 4, 7 and 9 are all mixed up at the top, and 2, 3, 5 and 8 are jumbled together in the centre. The whole cloud is centred around (0,0) with a spread of about 1 (I computed mean of μ as roughly (0.23, 0.19) and std around (1.07, 1.05)). This is entirely because the KL loss keeps pulling the latent codes towards $\mathcal{N}(0, I)$. The overlap happens for two reasons: two dimensions just aren’t enough to separate all ten digits, and the VAE never saw the labels during training.

19. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

Plot 7: Randomly Generated Images

Generate at least 25 images using randomly sampled latent vectors.

Display them in a grid.

Students should determine:

- whether the generated images resemble handwritten digits;
- whether digit shapes are recognizable;
- whether there is visible variation among generated samples; and
- whether some samples are ambiguous or unrealistic.

VAE: 25 images generated from $z \sim \mathcal{N}(0, I)$

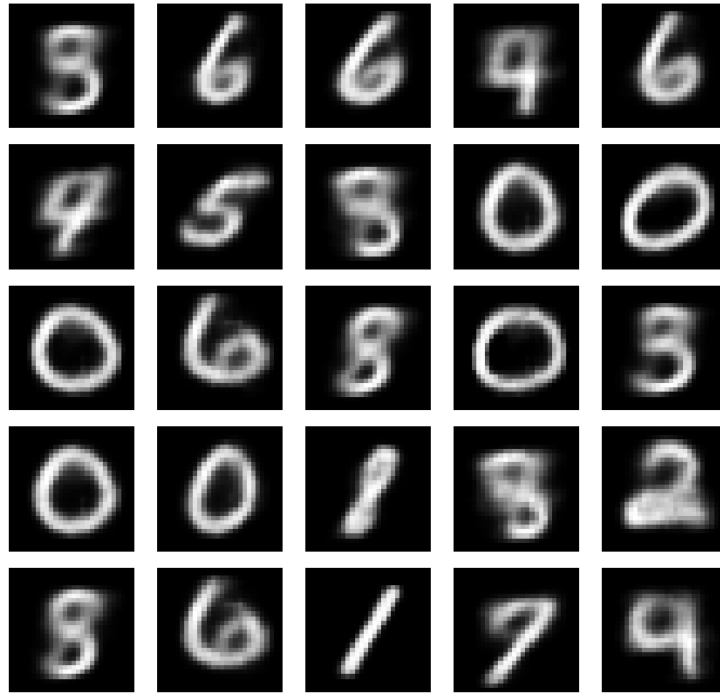


Figure 15: Plot 7: 25 images generated from random $z \sim \mathcal{N}(0, I)$.

VAE decoder manifold (z_1 left→right, z_2 bottom→top)

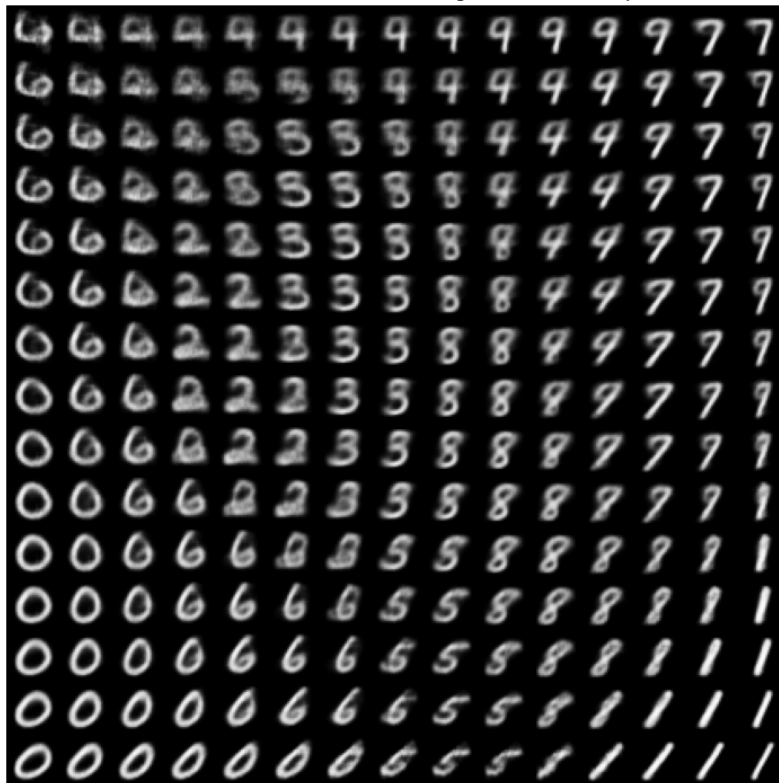


Figure 16: Extra: 15×15 grid of decoded images over the two latent dimensions.

Answer: A good chunk of the 25 generated images actually look like real handwritten digits. I can clearly make out 0s, 6s, 1s and 7s, plus a few 5s, 4s, 2s and 9s. There's genuine variety too — different digits, different slants, different thicknesses. But yeah, some of them are just blurry mush, like those fuzzy shapes that look halfway between a 3, a 5 and an 8. Those come from the regions in latent space where two digits overlap. So the VAE can definitely create new digits, but with only a 2-D latent space the quality isn't razor-sharp.

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \quad z_B.$$

Construct

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1].$$

Use values such as:

$$\alpha = 0, 0.1, 0.2, \dots, 1.$$

Decode every interpolated vector.

Plot 8: Latent Space Interpolation

Display the decoded images in sequence.

Required inference:

Describe whether the decoded image changes smoothly as the latent vector moves between two points.

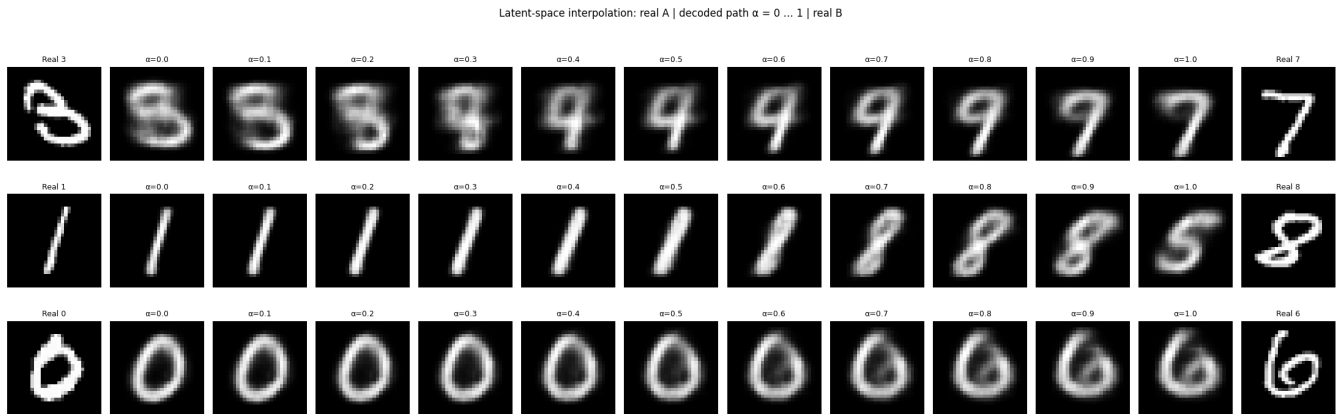


Figure 17: Plot 8: Latent-space interpolation for the pairs 3→7, 1→8 and 0→6 ($\alpha = 0, 0.1, \dots, 1$).

Answer: The figure shows 11 decoded images per pair, walking from $\alpha = 0$ to $\alpha = 1$. The transition is genuinely **smooth** — no sudden jumps anywhere. In the last row the 0 slowly opens up and grows a little loop at the bottom until it becomes a 6. In the middle row the 1 stays a 1 for a while and then gradually morphs into an 8-ish shape. In the top row the shape passes through a 4/9 kind of stage before settling on a 7. What I find interesting is that the in-between frames aren't random noise — they're actual digit-like images. That's real evidence that the VAE's latent space is continuous. One small caveat: at $\alpha = 0$ the decoded image isn't exactly the true input (the 3 comes out looking like a blurry 8), just because the 2-D code loses a lot of detail.

21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.012752	0.039208	0.8582	211,040
Convolutional AE	0.001011	0.009433	0.9904	74,497
Denoising CAE ($\sigma = 0.2$)	0.002956	0.016800	0.9630	74,497

The denoising CAE is tested on noisy test images and compared with the clean images.

For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

VAE Metric	Value
Reconstruction Loss	145.65
KL Loss	6.41
Total Loss	152.06
Test MSE	0.04147
Test MAE	0.09883
Mean SSIM	0.5386

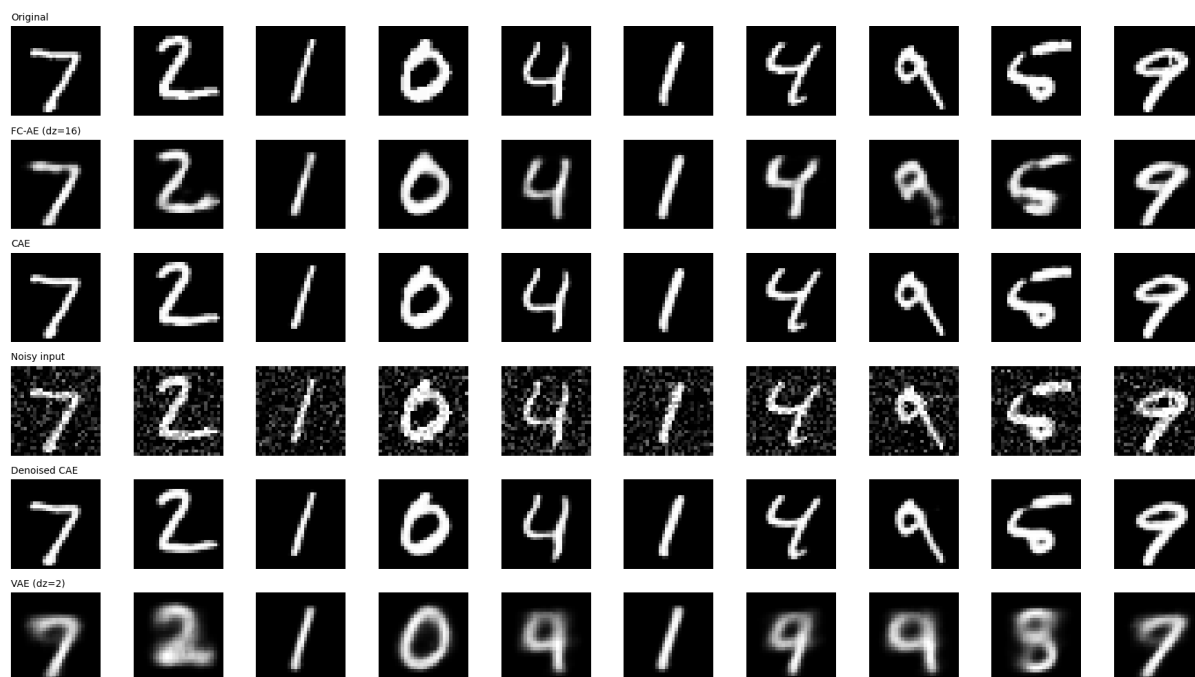


Figure 18: Comparison of all models on the same test digits (Original, FC-AE, CAE, Noisy input, Denoised CAE, VAE).

Answer: The CAE is the clear winner — lowest error, highest SSIM. The denoising CAE comes second, which is honestly impressive since it's working with corrupted input. Then the FC-AE, and finally the VAE with the worst reconstruction. The VAE loses here because it's only got a 2-number latent code, and on top of that the KL term is constantly pushing the code toward a normal distribution. But here's the trade-off: the VAE is the only one of these four that can **generate brand new images**. Also worth noting — the reconstruction and KL losses are per-image, so don't compare them directly to the MSE numbers.

22. Required Plots

The final report must contain the following plots:

1. Original image versus reconstructed image.
2. Training and validation loss for the basic autoencoder.
3. Fully connected AE versus convolutional AE reconstruction.
4. Clean versus noisy versus denoised images.
5. Noise level versus MSE/MAE/SSIM.
6. VAE latent-space visualization.
7. Randomly generated VAE images.
8. Latent-space interpolation.
9. Training/validation reconstruction loss for the VAE.
10. Reconstruction-error distribution on the test set.

Answer: All ten plots are in this report: (1) Figure 1, (2) Figure 2, (3) Figure 5, (4) Figure 7, (5) Figure 9, (6) Figure 14, (7) Figure 15, (8) Figure 17, (9) Figure 12, and (10) Figure 19 together with Figure 20. Plot 10 of Section 27 is Figure 22.

For plots containing multiple quantitative measures, use separate figures or clearly labeled axes rather than placing incompatible scales on one axis.

23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2.$$

Plot 9: Distribution of Reconstruction Errors

Plot a histogram of e_i over the test set.

Students should identify:

- the typical reconstruction-error range;
- whether the distribution is concentrated or spread out;
- possible high-error samples; and
- representative images corresponding to high reconstruction error.

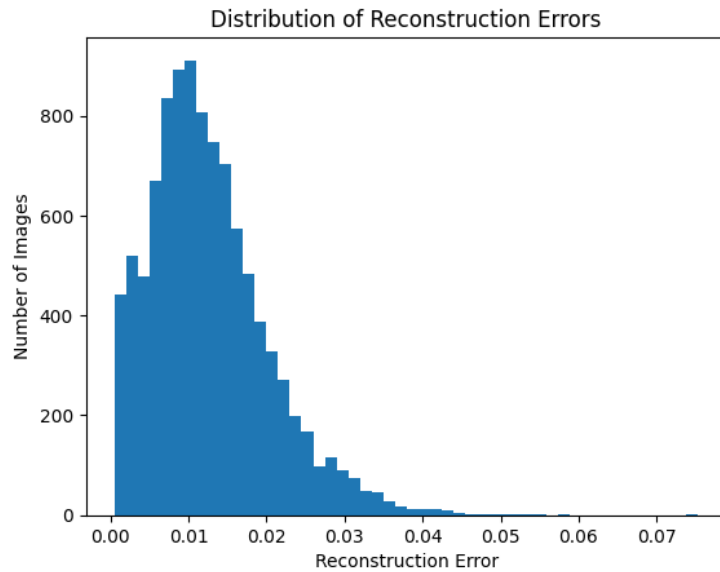


Figure 19: Plot 9: Distribution of per-image reconstruction error of the FC-AE on the test set.

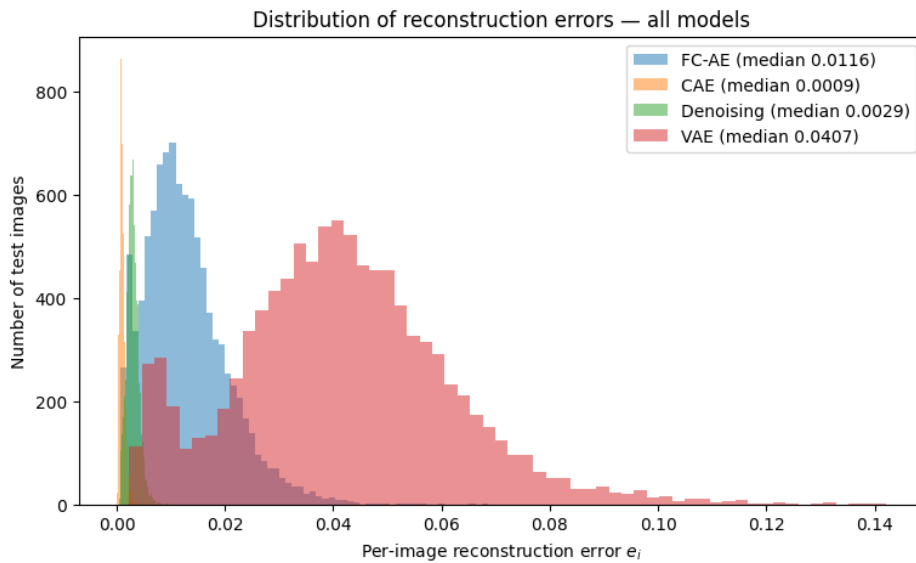


Figure 20: Distribution of per-image reconstruction error for all four models.

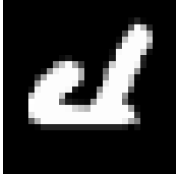
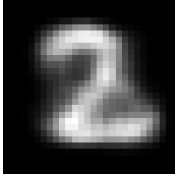
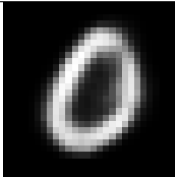
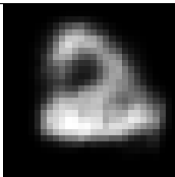
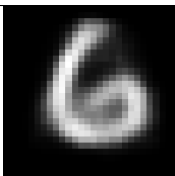
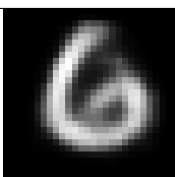
Answer: For the FC-AE, most images cluster between 0.005 and 0.02, with the tallest bar sitting right around 0.01. So the errors are pretty **concentrated** at low values. There's a long tail stretching out to the right though, and a handful of images hit errors up to about 0.075. Looking at the second plot, the CAE (median 0.0009) and the denoising CAE (median 0.0029) are both super narrow and pressed against zero, the FC-AE (median 0.0116) is

a bit wider, and the VAE (median 0.0407) is really spread out, reaching up to 0.14. The high-error images tend to be the weird, unusual ones — I'll show those next.

24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.

Display:

Sample	Original	Reconstruction
1 test #8000 digit 4 $e = 0.1422$		
2 test #3817 digit 2 $e = 0.1417$		
3 test #6161 digit 8 $e = 0.1398$		
4 test #1352 digit 2 $e = 0.1371$		
5 test #2802 digit 2 $e = 0.1320$		

Answer: This table shows the 5 worst test images for the VAE (where e is the mean squared error of the image). The VAE actually reconstructs them as completely different digits: a 4 becomes a 2, a 2 becomes a 0, an 8 becomes a 2, and two 2s turn into 6s. Why? Because these particular digits are written in really unusual ways — super thick, curly, or just strange-looking — so they don't resemble most of the training data. The 2-D latent code simply can't capture such odd shapes, so the decoder just defaults to the nearest common digit. For the FC-AE the worst offenders (Figure 21) were similarly messy 8s and odd 5s, with errors between 0.055 and 0.075, full of overlapping loops and extra strokes.

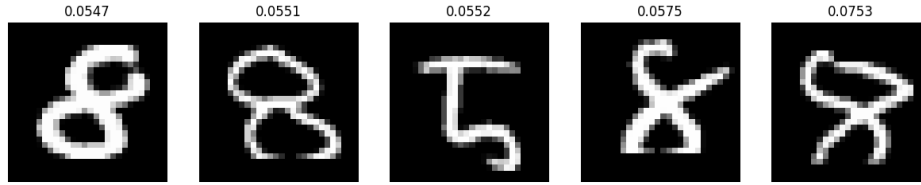


Figure 21: Five test images with the highest reconstruction error for the FC-AE (error shown above each image).

Students should provide a short explanation for why these images may be difficult to reconstruct.

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time
FC Autoencoder	0.012752	0.039208	0.8582	211,040	35.4 s
Conv. Autoencoder	0.001011	0.009433	0.9904	74,497	75.9 s
Denoising CAE ($\sigma = 0.2$)	0.002956	0.016800	0.9630	74,497	76.3 s
VAE ($d_z = 2$)	0.041470	0.098829	0.5386	134,165	114.8 s

The values must be obtained from the student's own execution.

Answer: These numbers are from my own runs. FC-AE trains the fastest. The CAE gives the best MSE and SSIM with the fewest parameters (74,497), so it's clearly the best pick if your goal is reconstruction. The VAE takes the longest because I trained it for 30 epochs (the others were 20) and its loss is more complicated. Also, note that the VAE's MSE and MAE are computed by decoding the mean μ of each image, not by sampling.

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn?

The following inferences are mandatory:

- effect of latent dimension on reconstruction;
- difference between fully connected and convolutional reconstruction;
- effect of Gaussian noise level;
- ability of the denoising autoencoder to recover image structure;
- characteristics of the VAE latent distribution;
- structure of the two-dimensional VAE latent space;
- quality and diversity of generated samples;
- smoothness of latent-space interpolation;
- relationship between reconstruction error and visual quality; and
- differences between deterministic and probabilistic latent representations.

Answer: Short inferences for the ten points:

1. **Latent dimension:** when I bump the latent dimension from 2 to 32, MSE drops from 0.0407 to 0.0100 and SSIM rises from 0.557 to 0.889. Obviously more numbers can hold more detail.
2. **FC vs convolutional:** CAE crushes the FC version here (MSE 0.0011 vs 0.0127), because convolution looks at neighbouring pixels together and shares filters across the image.
3. **Gaussian noise level:** higher σ means higher MSE/MAE and lower SSIM, simply because more pixel information gets destroyed.
4. **Recovering image structure:** even at $\sigma = 0.3$ the SSIM stays at 0.944, which tells me the denoising autoencoder really does recover the main shape of the digit.
5. **VAE latent distribution:** the latent codes hover around 0 with a spread near 1 — close to the prior $\mathcal{N}(0, I)$, thanks to the KL loss doing its job.
6. **2-D latent space:** same digits cluster together (0 and 1 are clearly separated), but 4/7/9 and 2/3/5/8 overlap heavily because two dimensions just aren't enough.
7. **Generated samples:** many samples look like genuine digits with real variety, but a good few are blurry blends, so quality is medium overall.
8. **Interpolation:** the digit changes smoothly from one class to another, which shows the latent space is continuous.

9. **Error and visual quality:** low error plus high SSIM lines up with sharp images (CAE); high error lines up with blurry or wrong digits (VAE). Pretty consistent.
10. **Deterministic vs probabilistic:** the normal AE gives one fixed point per image and reconstructs better, but the VAE gives a whole distribution — so it can generate new images and interpolate smoothly.

27. Additional Latent-Dimension Study

Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Report:

Latent Dimension	MSE	SSIM	Parameters
2	0.040652	0.5574	210,130
8	0.024273	0.7303	210,520
16	0.012752	0.8582	211,040
32	0.009995	0.8893	212,080

Plot 10: Latent Dimension vs. Reconstruction MSE

Students should explain the trade-off between compression and reconstruction quality.

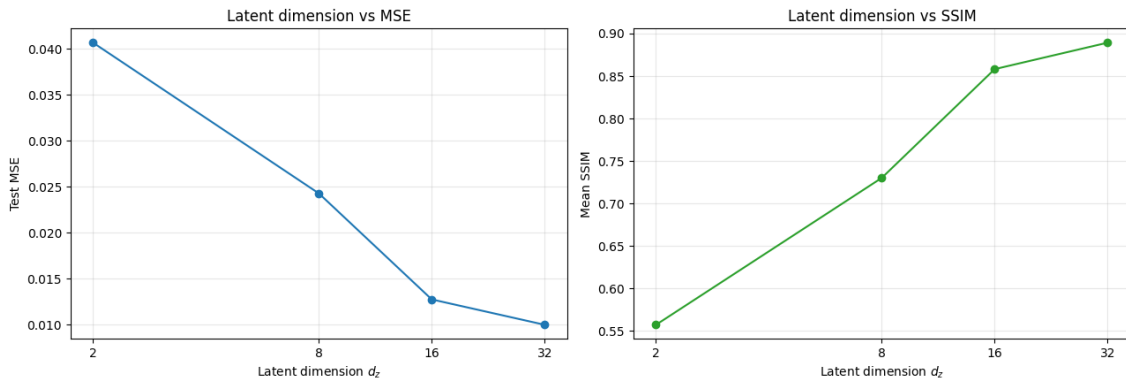


Figure 22: Plot 10: Latent dimension versus test MSE (left) and mean SSIM (right).

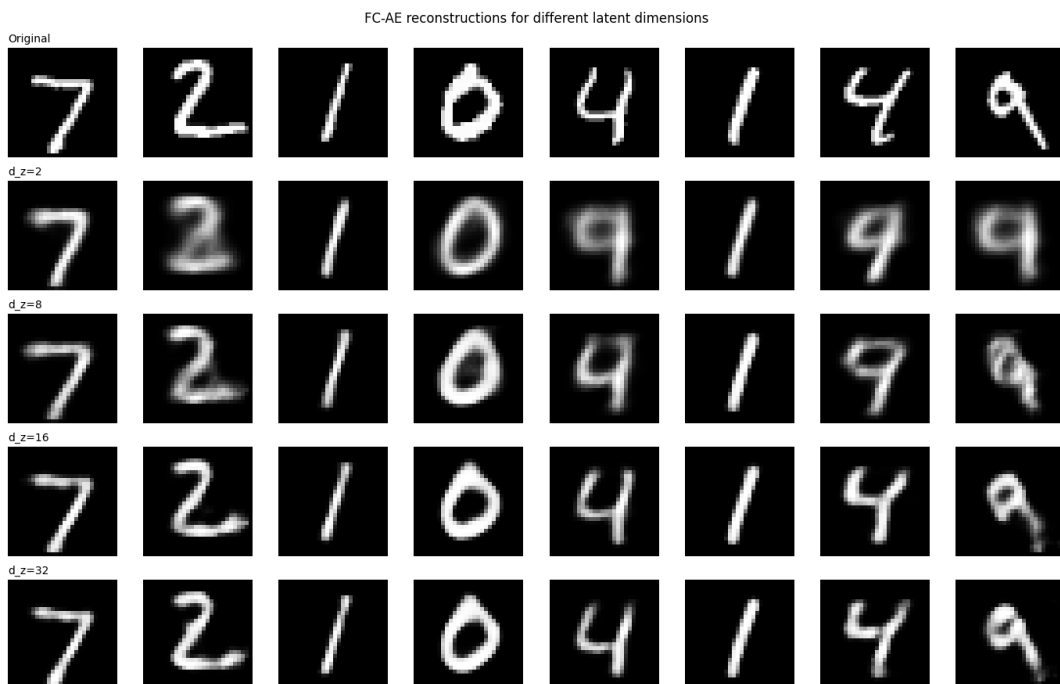


Figure 23: Reconstructions of the same 8 digits by the FC-AE for $d_z = 2, 8, 16, 32$.

Answer: As I increased the latent dimension, MSE dropped steadily (0.0407, 0.0243, 0.0128, 0.0100) and SSIM climbed (0.557, 0.730, 0.858, 0.889). The big jump is from 2 to 16 — after that, the gains get tiny, so you’re getting less and less bang for each extra number. Visually it’s clear too: at $d_z = 2$ the digits are super blurry (the 4 and the 9 basically look identical), at $d_z = 8$ it’s better but that last 9 is still bad, and by $d_z = 16$ and 32 they look almost indistinguishable from the originals. Funny thing is, the parameter count barely changes at all (all around 210,000). So yeah, there’s a real **trade-off** here: small latent vector = strong compression but poor quality, large latent vector = good quality but less compression.

28. Discussion Questions

1. **What is an autoencoder?**
It’s a neural network that’s trained to spit back out the same image you fed in. It manages this through a small middle layer.
2. **What is the purpose of the encoder?**
The encoder compresses the input — it turns the image into a short vector z .
3. **What is the purpose of the decoder?**
The decoder takes that short vector z and rebuilds the image back, $\hat{x}$.
4. **What is a latent representation?**
It’s that small compressed code z sitting in the middle of the autoencoder. It holds the important info about the image.
5. **Why is the latent representation usually smaller than the input?**
If it were big, the network could just cheat and copy the input. Making it small forces it to actually learn what matters.
6. **What happens when the latent dimension is too small?**
Too much information gets thrown away, so the output turns blurry or plain wrong. In my runs, $d_z = 2$ gave MSE 0.0407 and SSIM only 0.557.
7. **What happens when the latent dimension is increased?**
Reconstruction gets better (at $d_z = 32$ MSE was 0.0100), but you’re compressing less.
8. **Why can a convolutional autoencoder preserve spatial information more effectively?**
Because convolution filters look at neighbouring pixels together, so edges and shapes survive — plus the bottleneck is a feature map rather than a flat vector.
9. **Differentiate an autoencoder and a denoising autoencoder.**
A normal AE takes the clean image as both input and output. A denoising AE takes a noisy image as input, but the target stays the clean image.
10. **Why is the clean image used as the target in denoising?**
So the network is forced to learn how to strip out the noise. If you made the noisy image the target, it would just learn to copy the noise instead.
11. **What happens when the noise level is increased?**
MSE and MAE go up, SSIM goes down — so the denoised output gets a bit worse.
12. **Why are MSE, MAE and SSIM useful for image reconstruction?**
MSE and MAE tell you the average pixel error (MSE punishes big mistakes harder). SSIM tells you how structurally similar two images are, closer to how a human eye judges it.
13. **What is a Variational Autoencoder?**
It’s an autoencoder that outputs a distribution (a mean and a variance) for z instead of a single point, and that lets it generate fresh images.
14. **How does a VAE differ from a conventional autoencoder?**
A normal AE gives one fixed vector; a VAE gives μ and $\log \sigma^2$, samples z from them, and also has a KL loss term.
15. **Why does a VAE learn a distribution instead of a single latent vector?**
So the latent space becomes smooth and continuous, and nearby points decode into similar images.
16. **What is the reparameterization trick?**
It rewrites $z = \mu + \sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$. The randomness gets pushed onto ϵ so gradients can still flow back through μ and σ .
17. **Why is ϵ sampled from a standard normal distribution?**
Because it’s a simple fixed noise with mean 0 and variance 1 and no parameters — all the learning happens in μ and σ .
18. **What is the role of KL divergence in the VAE loss?**
It keeps the learned distribution close to the prior $\mathcal{N}(0, I)$, which regularizes the latent space so we can sample from it.
19. **What is the prior distribution used in the VAE?**
The standard normal, $p(z) = \mathcal{N}(0, I)$.

20. **Why is a two-dimensional latent space useful for visualization?**

Because you can just plot it as a graph with z_1 and z_2 and visually see how digits cluster or overlap.

21. **What does latent-space interpolation demonstrate?**

It shows the latent space is smooth — moving slowly between two points slowly morphs one digit into another.

22. **Why can a VAE generate new images?**

Because the latent codes get pushed toward $\mathcal{N}(0, I)$, so sampling a random z from that and decoding gives you a brand new image.

23. **Why might some VAE-generated images be unclear?**

With only 2 dimensions lots of digits overlap, so a point between two digits gives a mixed shape. Plus the VAE loss naturally produces slightly blurry outputs.

24. **What is the difference between reconstruction and generation?**

In reconstruction you encode a real image and decode it back. In generation you sample a random z from the prior and decode it — no input image involved.

25. **Why should test images remain unseen during training?**

So you can honestly check how well the model handles new data. If it's already seen them, you can't detect overfitting.

29. Additional Exercises

1. Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.
2. Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.
3. Train the denoising autoencoder using two different noise levels and compare SSIM.
4. Replace the convolutional decoder with transposed convolutions and compare the reconstructions.
5. Implement a VAE with latent dimension 2 and 8 and compare the latent representations.
6. Investigate the effect of increasing the KL-loss contribution.
7. Generate a larger grid of VAE samples and discuss diversity.
8. Select two digit regions in the latent space and perform interpolation.

Answer: Exercise 1: Effect of Latent Dimension on Convolutional Autoencoder Reconstruction

The convolutional autoencoder was trained using latent dimensions of 4, 8, 16, and 32. The reconstruction performance was evaluated using Mean Squared Error (MSE), Mean Absolute Error (MAE), and Structural Similarity Index Measure (SSIM).

Latent Dim.	MSE	MAE	SSIM	Parameters
4	0.02933	0.07254	0.69390	65,797
8	0.01603	0.04578	0.82010	90,889
16	0.00725	0.02671	0.92464	141,073
32	0.00361	0.01797	0.96161	241,441

The results show that reconstruction quality improves as the latent dimension increases. Increasing the latent dimension from 4 to 32 reduces MSE from 0.02933 to 0.00361 and MAE from 0.07254 to 0.01797. SSIM increases from 0.69390 to 0.96161, indicating improved structural similarity between the reconstructed and original images. However, the parameter count also increases from 65,797 to 241,441. Therefore, a larger latent representation preserves more information and improves reconstruction quality, but requires a model with more parameters.

Exercise 2: Comparison of Gaussian and Salt-and-Pepper Noise

The same denoising architecture was evaluated using Gaussian noise and salt-and-pepper noise. At the highest tested noise levels, Gaussian noise with $\sigma = 0.3$ achieved an SSIM of 0.944, while salt-and-pepper noise with probability $p = 0.2$ achieved an SSIM of 0.957.

The salt-and-pepper noise experiment produced an SSIM that was 0.013 higher than that of Gaussian noise under the reported settings. This indicates that the architecture reconstructed the tested salt-and-pepper-corrupted images with slightly higher structural similarity. The result applies to the specific noise levels evaluated and does not necessarily imply that salt-and-pepper noise is easier to remove in every setting.

Exercise 3: Effect of Noise Level on Denoising Performance

The denoising autoencoder was evaluated at three different noise levels, as reported in Section 14. SSIM was used to compare the structural similarity of the reconstructed images with the original images.

The comparison illustrates how the amount of input corruption affects reconstruction quality. SSIM values should be interpreted alongside the corresponding noise levels to determine how well the model preserves image structure as noise increases. The numerical results from Section 14 are retained in that section rather than duplicated here.

Exercise 4: Comparison of Decoder Architectures

The convolutional autoencoder was evaluated using a conventional upsampling decoder and a transposed-convolution decoder.

Decoder	MSE	MAE	SSIM	Time (s)
Upsampling	0.001098	0.009806	0.989259	67.00
Transposed Convolution	0.001054	0.009477	0.990062	54.23

Both architectures achieved high reconstruction quality and used the same number of parameters, 74,497. The transposed-convolution decoder achieved a slightly lower MSE and MAE, together with a slightly higher SSIM. Its recorded execution time was approximately 54.23 seconds, compared with 67.00 seconds for conventional upsampling. Thus, in this experiment, the transposed-convolution decoder achieved marginally better reconstruction metrics and a shorter recorded execution time.

Exercise 5: Comparison of VAE Latent Dimensions

Variational autoencoders (VAEs) with latent dimensions of 2 and 8 were trained and evaluated.

Latent Dim.	MSE	MAE	SSIM	KL Total	Active Dims.
2	0.04294	0.10100	0.52000	5.96991	2
8	0.03493	0.08518	0.61620	8.31797	3

The VAE with latent dimension 8 achieved better reconstruction performance than the VAE with latent dimension 2. Its MSE decreased from 0.04294 to 0.03493, while its SSIM increased from 0.52000 to 0.61620. However, the eight-dimensional model had only three active latent dimensions according to the reported activity analysis. Several dimensions had very small mean standard deviations and KL contributions, suggesting that the model did not make effective use of every available latent dimension. The PCA analysis of the eight-dimensional latent representation reported variance proportions of 0.52515 and 0.30998 for the first two principal components. These results suggest that increasing the latent dimension provides additional representational capacity, although not all dimensions necessarily contribute meaningfully to the learned representation.

Exercise 6: Effect of Increasing the KL-Loss Contribution

The effect of the KL-loss weight β was investigated using values of 0.1, 1, 5, and 20.

β	MSE	MAE	SSIM	KL Total	Gen. Classes
0.1	0.04060	0.09649	0.55127	11.05655	8
1	0.04136	0.09798	0.53614	6.15845	10
5	0.04968	0.11458	0.42359	2.62777	8
20	0.05777	0.13463	0.27258	0.63867	4

As the KL-loss weight increases, the total KL divergence decreases from 11.05655 at $\beta = 0.1$ to 0.63867 at $\beta = 20$. At the same time, reconstruction quality deteriorates: MSE increases from 0.04060 to 0.05777, and SSIM decreases from 0.55127 to 0.27258. The number of predicted digit classes among generated samples also changes, with all ten classes represented at $\beta = 1$, compared with four classes at $\beta = 20$.

These results illustrate the trade-off between reconstruction accuracy and latent-space regularization. A larger KL-loss weight encourages the latent distribution to remain closer to the prior, but excessive regularization can reduce the information retained in the latent representation and impair reconstruction quality. In this experiment, the highest KL-loss weight produced the weakest reconstruction metrics.

Exercise 7: Diversity of Generated VAE Samples

A larger 15×15 grid of generated VAE samples was produced, as shown in Figure 16. The generated samples were evaluated using a digit classifier.

Latent Dim.	Classes Present	Mean Confidence	Low-Confidence Samples
2	10	0.85237	29
8	10	0.86530	33

Both latent configurations generated samples classified into all ten digit classes. The eight-dimensional model achieved a slightly higher mean classifier confidence of 0.86530, compared with 0.85237 for the two-dimensional model. However, it also produced 33 low-confidence samples, compared with 29 for the two-dimensional model. The class counts varied between the two configurations, indicating that the generated samples were not distributed uniformly across all digit classes. Overall, the grid demonstrates diversity across digit categories, while the confidence results indicate that some generated images may be ambiguous or poorly formed. Classifier confidence is used here as an indicator of recognizability rather than a direct measure of visual quality.

Exercise 8: Interpolation in Latent Space

Latent-space interpolation was performed between the digit pairs $3 \rightarrow 7$, $1 \rightarrow 8$, and $0 \rightarrow 6$. The resulting interpolated images are presented in Figure 17.

Interpolation explores the path between two digit representations in the latent space. Intermediate samples can reveal whether the learned representation changes gradually between the selected digits. Smooth transitions suggest that nearby latent points can decode to meaningful variations in digit shape, whereas abrupt changes or ambiguous intermediate images may indicate limitations in the learned latent geometry. The three selected digit pairs provide examples for examining how the VAE represents relationships between different digit classes.

Overall Inference

The additional experiments demonstrate that autoencoder performance depends on the size and structure of the latent representation, the decoder architecture, and the choice of loss function. Increasing the convolutional autoencoder's latent dimension improved reconstruction quality but also increased the number of parameters. The transposed-convolution decoder produced slightly better reconstruction metrics and a shorter recorded execution time than the conventional upsampling decoder in the tested configuration.

For the VAE, increasing the latent dimension from 2 to 8 improved reconstruction metrics, although only three of the eight latent dimensions were identified as active. Increasing the KL-loss contribution reduced KL divergence but worsened reconstruction metrics at larger weights. The generated sample grid covered all ten digit classes for both tested latent dimensions, and interpolation provided a way to examine relationships between digit representations in the latent space. Taken together, these experiments illustrate the trade-offs between reconstruction fidelity, latent-space regularization, representational capacity, and generative diversity.

30. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

Image $\rightarrow$ Encoder $\rightarrow$ Latent Representation $\rightarrow$ Decoder $\rightarrow$ Reconstructed Image

They should also demonstrate denoising:

Clean Image $\rightarrow$ Noise $\rightarrow$ Denoising Autoencoder $\rightarrow$ Recovered Image

and variational generation:

Image $\rightarrow (\mu, \sigma) \rightarrow z \rightarrow$ Decoder $\rightarrow$ Generated/Reconstructed Image

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

Answer: Conclusion. I trained four flavors of autoencoders on MNIST. The fully connected one preserved digit shapes but produced blurry outputs (MSE 0.0127, SSIM 0.86). The convolutional one was dramatically better (MSE 0.0011, SSIM 0.99) and did it with fewer parameters. The denoising autoencoder cleaned up both Gaussian and salt-and-pepper noise really well, and quality only dropped gradually as noise increased. The VAE had noticeably worse reconstruction (SSIM 0.54) but it could actually generate new digits, and its latent space was smooth — the interpolation experiment proved that nicely. And as a general takeaway: a bigger latent dimension gives better reconstruction but less compression.

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.
3. Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, "MNIST Handwritten Digit Database."
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. scikit-image Documentation: <https://scikit-image.org>

32. Source Code

The complete source code is available at:

<https://github.com/guru963/deeplearning/tree/main/lab7>